Information-Dynamical Design of a Self-Consistent Multitask Learning Processor

A revised theoretical note on circulation, recursion, robustness, pruning, reasoning, and subgraph emergence

generated by ChatGPT5.5 directed by Yunju Lee

June 2026

1 Introduction

The purpose of this note is to formulate the design logic for an information processor that can manage multiple tasks, remain robust against inconsistent inputs, and preserve self-consistency while reducing its dependence on externally supplied input-answer pairs. The target object is therefore not merely a classifier, nor merely a large approximator. It is a finite-capacity learning system whose internal organization must support four operations at once: acquisition of task-relevant information, validation of internally generated constraints, rejection or suppression of incompatible information, and conditional routing among different task structures.

The central problem can be expressed as follows. A supervised machine receives external pairs (X, C), where X is an input and C is the correct answer. Its output parameters θ are adjusted so that $Y = F_\theta(X)$ carries information about C. External supervision is expensive, so an efficient learning processor should not require an unlimited supply of externally labeled pairs. A second parameter system γ may therefore generate candidate input-answer pairs after an initial supervised stage. However, such internally generated pairs are useful only if they are validated against the already acquired task structure and then returned to the update of θ and γ. This creates a circulation of information rather than a one-way supervised feed.

The essential expressivity point is only this: even if a sufficiently large network can approximate many functions, approximation capacity alone does not explain how the correct function is selected, how false constraints are rejected, or how previously acquired structure is preserved.

The design logic is consequently inevitable from the problem setting. A useful processor separates proposal, validation, and update; it measures incremental information flow rather than raw activity; it preserves higher-order groups when the task requires group information; it uses finite capacity margins as signals of inconsistency; it repairs reasoning gaps by active bridge generation; and it uses a shared backbone with context-dependent routing for multitask operation. These principles are not additional decorations. They follow from the requirement that the same finite information processor must learn efficiently, validate itself, remain robust, and solve more than one kind of problem.

Deduction in one line

Limited supervision plus finite capacity forces validated circulation; finite capacity plus circulation creates competition against inconsistent flows; graph topology plus competition makes pruning and context-dependent subgraphs useful.

The full deduction chain is the following. The task is represented by (X, C) and success by $I(C; Y)$;

the machine is a finite-capacity graph-structured information processor; operational flow through a feedforward graph is recursively supported by directed paths and corrected by higher-order terms at mergers; ordinary supervised learning updates θ by external pairs; label-efficient self-learning introduces a generator γ of candidate pairs; generated pairs are not automatically valid, so validation must use already acquired structure; accepted generated pairs return to update θ and γ, forming a validated circulation; finite entropy capacity at merging vertices makes flows compete; a strong validated circulation leaves little residual capacity for incompatible external information; pruning removes unnecessary routes and sharpens this capacity constraint; reasoning gaps are local missing routes in a sequential information graph, so repairing a gap inserts bridge information into the circulation; and multiple task types activate partially shared and partially exclusive effective subgraphs. Consequently, the processor should not be designed as a passive feedforward approximator alone, but as a proposal-validation-update system with finite-capacity competition, pruning or sparsification of irrelevant routes, reasoning repair, and context-dependent routing.

2 Compressed problem setting and feedforward recursion

2.1 Learning as acquisition of predefined task information

Let X denote an input random variable and let C denote the correct answer associated with the input. In an ideal noise-free classification problem,

$$C = f^*(X), \qquad H(C \mid X) = 0, \tag{1}$$

so that

$$I(C; X) = H(C). \tag{2}$$

In a noisy or ambiguous problem,

$$I(C; X) = H(C) - H(C \mid X) < H(C). \tag{3}$$

At learning step s, the machine output is

$$Y_s = F_{h_s}(X), \tag{4}$$

where h_s denotes the current machine state or parameterization. The amount of task information expressed by the output is

$$K_s := I(C; Y_s). \tag{5}$$

A perfect noise-free learner satisfies $K_s = H(C)$ after training. A partial learner satisfies $0 < K_s < H(C)$.

The slow learning-scale transfer entropy and reversed transfer entropy are

$$\mathcal{T}^{\text{learn}}_{C\to Y}(s) := I(C; Y_{s+1} \mid Y_s), \tag{6}$$

$$\mathcal{T}^{\text{rev,learn}}_{C\to Y}(s) := I(C; Y_s \mid Y_{s+1}). \tag{7}$$

Since C itself is fixed with respect to the learning index, the chain rule gives

$$\Delta_s I(C; Y_s) = \mathcal{T}^{\text{learn}}_{C\to Y}(s) - \mathcal{T}^{\text{rev,learn}}_{C\to Y}(s). \tag{8}$$

Thus net learning is the balance between newly expressed task information and task information lost after an update.

2.2 Two timescales and the parameter graph

The black-box map F_{h_s} is resolved into a graph operating on two timescales. The slow index s labels learning updates. The fast index t labels operational propagation while the weights are frozen. Write

$$h_s(t) = \big(\sigma(t;s), W(s)\big), \tag{9}$$

where

$$\sigma(t;s) = \{\sigma_v(t;s)\}_{v\in\mathcal{V}} \tag{10}$$

collects vertex states and $W(s)$ encodes a weighted directed graph or directed hypergraph. For an ordinary directed graph,

$$W_{uv}(s) = \text{weight of the edge } u \to v. \tag{11}$$

For a hypergraph,

$$W_{A\to v}(s) = \text{weight of the hyperarc } A \to v, \qquad A \subseteq \mathcal{V}. \tag{12}$$

The simplest analysis assumes a fixed support $\mathcal{E} = \operatorname{supp} W(s)$, while learning changes the weights:

$$W(s) \longrightarrow W(s+1). \tag{13}$$

At fixed s, the operational dynamics are

$$\sigma(t+1;s) = G_{W(s)}\big(\sigma(t;s), \xi(t;s)\big), \tag{14}$$

where ξ represents optional stochasticity. The input determines the initial or boundary state,

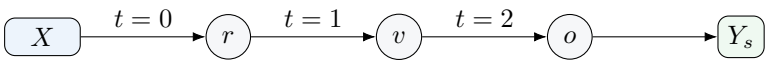
$$\sigma(0;s) = E(X), \tag{15}$$

and the output is read from an output vertex or layer:

$$Y_s = R(\sigma(T;s)). \tag{16}$$

Therefore

$$F_{h_s} = R \circ G^T_{W(s)} \circ E. \tag{17}$$



$W(s)$ frozen during operational propagation; $W(s) \to W(s+1)$ during learning

2.3 Operational transfer entropy and multiple transfer entropy

For an edge $x \to y$, suppressing the explicit learning index s when no confusion is possible, define the operational transfer entropy

$$\mathcal{T}_{x\to y}(t;s) := I\big(\sigma_y(t+1;s); \sigma_x(t;s) \mid \sigma_y(t;s)\big), \tag{18}$$

and the operational reversed transfer entropy

$$\mathcal{T}^{\mathrm{rev}}_{x\to y}(t;s) := I\big(\sigma_y(t;s); \sigma_x(t+1;s) \mid \sigma_y(t+1;s)\big). \tag{19}$$

For a source set $A = \{x_1, \dots, x_n\}$, define the nth-order multiple transfer entropy by the interaction-information convention

$$\mathcal{T}^n_{A\to y}(t;s) := I^n_{\sigma_{x_1}(t;s):\cdots:\sigma_{x_n}(t;s):\sigma_y(t+1;s)|\sigma_y(t;s)}. \tag{20}$$

The corresponding multiple reversed transfer entropy is

$$\mathcal{T}^{n,\mathrm{rev}}_{A\to y}(t;s) := I^{n}_{\sigma_{x_1}(t+1;s):\cdots:\sigma_{x_n}(t+1;s):\sigma_y(t;s)|\sigma_y(t+1;s)}. \tag{21}$$

For $n = 1$, these reduce to ordinary TE and ordinary reversed TE.

For a dynamic-neighbor set K_y, define the multi-order correction

$$E_{K_y\to y} := \sum_{n=2}^{|K_y|}(-1)^n \sum_{\substack{A\subseteq K_y\\ |A|=n}} \left[\mathcal{T}^{n}_{A\to y} - \mathcal{T}^{n,\mathrm{rev}}_{A\to y}\right]. \tag{22}$$

This term collects the higher-order informational correction that is missed by a purely edgewise sum.

2.4 Local operational balance

For an edge $j \to i$, let K_j be the dynamic-neighbor set of j. The local operational balance has the form

$$\Delta_t \mathcal{T}_{j\to i} = \sum_{\eta\in K_j\setminus\{i\}} \left[\mathcal{T}_{\eta\to j} - \mathcal{T}^{\mathrm{rev}}_{\eta\to j}\right] - \left[\mathcal{T}_{j\to i} - \mathcal{T}^{\mathrm{rev}}_{j\to i}\right] - E_{K_j\to j} \tag{23}$$

$$+ \alpha_1^{K_j\to j} - \alpha_2^{ji} - \alpha_3^{j\to i}. \tag{24}$$

All terms are evaluated at $(t; s)$. The displayed structure is

$$\text{change of outgoing TE} = \text{incoming net flow} - \text{existing outgoing net flow} - \text{multi-order correction} + \text{controls}. \tag{25}$$

The variables $\alpha_1, \alpha_2, \alpha_3$ encode features not determined by the displayed transfer entropies alone, such as internal uncertainty, mutual-information coupling, and edge-specific control.

2.5 Feedforward topology and recursive calculation

Assume the operational graph is a directed acyclic graph with layers

$$L_0 \longrightarrow L_1 \longrightarrow \cdots \longrightarrow L_D. \tag{26}$$

Let $R \subseteq L_0$ be receiving vertices connected to the input X, and let $O \subseteq L_D$ be output vertices. Immediately before an outgoing channel $j \to i$ is activated for the first time, assume

$$\mathcal{T}_{j\to i}(t;s) = 0, \qquad \mathcal{T}^{\mathrm{rev}}_{j\to i}(t;s) = 0. \tag{27}$$

Assume also that reverse-direction operational flows and displayed controls are negligible at the first activation:

$$\mathcal{T}^{\mathrm{rev}} = 0, \qquad \alpha_1 = \alpha_2 = \alpha_3 = 0. \tag{28}$$

Then the previous silence of the downstream channel gives

$$\Delta_t \mathcal{T}_{j\to i}(t;s) = \mathcal{T}_{j\to i}(t+1;s). \tag{29}$$

Substituting this into Eq. (24) yields the first-activation recursion

$$\mathcal{T}_{j\to i}(t+1;s) = \sum_{p\in \mathrm{Pa}(j)} \mathcal{T}_{p\to j}(t;s) - E_{\mathrm{Pa}(j)\to j}(t;s). \tag{30}$$

Equivalently,

$$\mathcal{T}_{j\to i}(t+1;s)=\sum_{n=1}^{|\mathrm{Pa}(j)|}(-1)^{n+1}\sum_{\substack{A\subseteq\mathrm{Pa}(j)\\|A|=n}}\mathcal{T}^{n}_{A\to j}(t;s). \tag{31}$$

Equation (31) is the main feedforward recursion. An outgoing ordinary TE is generated by the inclusion-exclusion combination of incoming ordinary and multiple TEs.

If j has one parent p, then $E_{\mathrm{Pa}(j)\to j}=0$ and

$$\mathcal{T}_{j\to i}(t+1;s)=\mathcal{T}_{p\to j}(t;s). \tag{32}$$

If $\mathrm{Pa}(j)=\{p_1,p_2\}$, then

$$\mathcal{T}_{j\to i}(t+1;s)=\mathcal{T}_{p_1\to j}(t;s)+\mathcal{T}_{p_2\to j}(t;s)-\mathcal{T}^{2}_{\{p_1,p_2\}\to j}(t;s). \tag{33}$$

Thus the second-order multiple TE corrects the naive sum of two incoming ordinary TEs.

2.6 Graph-to-output operational flow

Define the effective operational influx received by a vertex v:

$$\Phi_v(s):=\sum_{n=1}^{|\mathrm{Pa}(v)|}(-1)^{n+1}\sum_{\substack{A\subseteq\mathrm{Pa}(v)\\|A|=n}}\mathcal{T}^{n}_{A\to v}(s). \tag{34}$$

For a receiving vertex $r\in R$ with a single input source X,

$$\Phi_r(s)=\mathcal{T}_{X\to r}(s). \tag{35}$$

For an internal vertex v,

$$\Phi_v(s)=\sum_{p\in\mathrm{Pa}(v)}\Phi_p(s)-E_{\mathrm{Pa}(v)\to v}(s). \tag{36}$$

For each child $w\in\mathrm{Ch}(v)$, the outgoing operational TE at first activation is

$$\mathcal{T}_{v\to w}(s)=\Phi_v(s). \tag{37}$$

Therefore, for an output vertex $o\in O$, the operational influx delivered by the graph is obtained recursively from boundary flows and merger corrections:

$$\Phi_o(s)=R_{G,o}\left[\{\mathcal{T}_{X\to r}(s)\}_{r\in R},\{E_{\mathrm{Pa}(v)\to v}(s)\}_{v\in\mathrm{Anc}(o)}\right]. \tag{38}$$

This recursive expression is preferable to an unrestricted global path sum. If information is copied into several branches and later recombined, a naive path sum counts the same information repeatedly. The local corrections $E_{\mathrm{Pa}(v)\to v}$ retain the informational structure at each merger.

3 Learning circulation and the unified deduction chain

3.1 Why a one-way supervised feed is insufficient

The ordinary supervised loop is one-way:

$$(X,C)_{\mathrm{ext}}\longrightarrow\text{update of }\theta\longrightarrow Y=F_\theta(X). \tag{39}$$

This loop can train a model, but it does not by itself explain autonomous improvement after external labels become scarce. Once external supervision is reduced, the machine must either stop improving or obtain new constraints from another source. The proposed source is an internal generator with parameters γ.

Let

$$(\widehat{X}, \widehat{C}) = G_\gamma(S_\theta, \zeta) \tag{40}$$

be an internally generated candidate pair, where S_θ denotes the current internal state of the learned processor and ζ denotes optional sampling noise. The pair can update θ only if it passes a validation operation

$$V_{\theta,\gamma}(\widehat{X}, \widehat{C}) \in \{0, 1\} \tag{41}$$

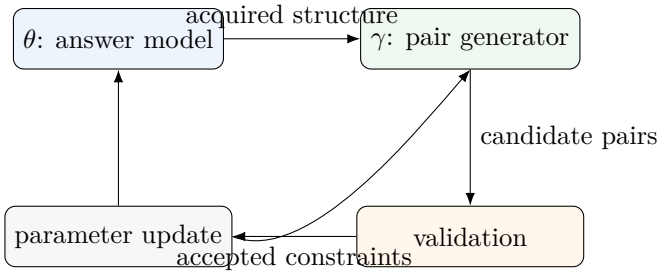
or a graded validity score. The candidate pair must then be returned to the update rules

$$\theta_{s+1} = U_\theta(\theta_s; \widehat{X}, \widehat{C}, V_{\theta,\gamma}), \tag{42}$$

$$\gamma_{s+1} = U_\gamma(\gamma_s; \widehat{X}, \widehat{C}, V_{\theta,\gamma}). \tag{43}$$

The structure is therefore a circulation:

$$\theta \longrightarrow \gamma\text{-generated candidate constraints} \longrightarrow \text{validation by acquired structure} \longrightarrow (\theta, \gamma) \text{ update.} \tag{44}$$



3.2 Necessity of circulation under the stated goal

The claim is conditional. If the goal is merely to fit a finite externally supplied dataset, circulation is not structurally necessary. If the goal is label-efficient self-learning, the situation changes.

Assume:

1. external supervision is finite or costly;
2. the machine is expected to continue producing task-relevant improvements;
3. internally generated constraints may be false;
4. false constraints must not freely update the answer model.

Then any useful internal constraint must pass through a validation channel before it modifies the model. Since validation depends on the already learned structure and the result of validation modifies the later learned structure, the learning system contains a closed informational route. Therefore, under these assumptions, efficient self-learning requires a validated circulation.

3.3 Connection between operational flow and learning flow

Operational flow propagates information through a fixed graph. Learning flow modifies the graph. The two are related because useful parameter updates are those that increase future operational

task information. Let $\Phi_o(s)$ be the operational influx delivered to the output at learning step s. A learning update is task-relevant when

$$\Delta_s I(C; Y_s) > 0 \tag{45}$$

and when this increase can be attributed to a change of operational routes or weights that increases $\Phi_o(s+1)$ relative to $\Phi_o(s)$. In schematic form,

$$\mathcal{T}^{\text{learn}}_{C\to\theta}(s) \quad \Longrightarrow \quad \Delta_s W(s) \quad \Longrightarrow \quad \Delta_s \Phi_o(s). \tag{46}$$

Thus learning TE is not the same quantity as operational TE, but successful learning should reorganize the operational TE recursion so that more task-relevant flow reaches the output.

4 Finite entropy capacity and competition at merging vertices

4.1 Capacity bound

Let v be a vertex with state Z_v and update Z'_v. Suppose a source set A transmits information to the update of v. The joint operational flow satisfies

$$I(A; Z'_v \mid Z_v) \le H(Z'_v \mid Z_v). \tag{47}$$

Define the local entropy capacity

$$C_v := H(Z'_v \mid Z_v). \tag{48}$$

Then

$$\mathcal{T}^{\text{joint}}_{A\to v} \le C_v. \tag{49}$$

This is the basic finite-capacity constraint. A vertex cannot receive more conditional information about its next state than the conditional entropy available in that update.

4.2 Residual capacity for a new flow

Let A denote an already established internal circulation arriving at v, and let B denote a newly activated external or competing source. The chain rule gives

$$I(A, B; Z'_v \mid Z_v) = I(A; Z'_v \mid Z_v) + I(B; Z'_v \mid Z_v, A). \tag{50}$$

Combining this with Eq. (49) yields

$$I(B; Z'_v \mid Z_v, A) \le C_v - I(A; Z'_v \mid Z_v). \tag{51}$$

If the internal circulation nearly saturates the local capacity,

$$I(A; Z'_v \mid Z_v) \ge C_v - \varepsilon, \tag{52}$$

then

$$I(B; Z'_v \mid Z_v, A) \le \varepsilon. \tag{53}$$

Thus a strong validated internal circulation suppresses the genuinely new conditional information that an additional source can insert at the same merging vertex.

4.3 Interpretation as self-consistency

The residual-capacity inequality should not be read as saying that all new external information is false. It says something more precise. If a new source B is consistent with the already established source A, then the new information may be redundant with A and may not require much residual capacity. If B is incompatible with A, then it must introduce genuinely new conditional information relative to A. That information is bounded by the residual capacity. Near saturation, the admissible margin is small.

Therefore, a finite-capacity circulation can act as a self-consistency mechanism. It does not identify truth metaphysically. It creates a dynamical test: an input that competes strongly with validated internal circulation must overcome a small residual information margin, while a compatible input can enter as redundant or confirmatory information.

Robustness criterion

A vertex or subgraph is robust when task-relevant circulation occupies enough conditional entropy capacity to make inconsistent new flows expensive, while still leaving a controlled margin for genuine compatible updates.

4.4 Role of synergistic exceptions

The additive capacity picture is exact when written in terms of joint conditional information. However, ordinary edgewise flows may be misleading in the presence of strong synergy. If A and B jointly determine Z'_v but neither source is informative alone, then ordinary pairwise TE can be small while joint TE is large. Therefore, robust design must monitor higher-order terms at merging vertices. The information-flow hyperarc $\{A, B\} \to v$ is then the natural representation of the relevant flow.

5 Redundancy analysis and pruning as requirements of robustness

5.1 Correlated inputs and redundant flow

Suppose the input decomposes into components $X = (X_1, X_2)$ with

$$I(X_1; X_2) > 0. \tag{54}$$

If both components are routed to the same target, the total joint information is

$$I(X_1, X_2; Z'_v \mid Z_v) = I(X_1; Z'_v \mid Z_v) + I(X_2; Z'_v \mid Z_v) \tag{55}$$
$$- I(X_1; X_2; Z'_v \mid Z_v). \tag{56}$$

When the interaction term is positive, the two components contain redundant information about the target update. A naive sum of edgewise flows overestimates the genuinely new information delivered to the vertex.

Redundancy is not always harmful. Redundant routes can increase stability against noise. However, under finite entropy capacity, redundant routes can also consume degrees of freedom that could otherwise be used for task-relevant distinctions. The question is therefore not whether redundancy exists, but whether it is task-relevant.

5.2 Unnecessary vertices

Call a vertex u unnecessary for a task if removing u leaves the task information at the output essentially unchanged:

$$I(C; Y_{\mathcal{G}}) - I(C; Y_{\mathcal{G}\setminus u}) \leq \delta, \tag{57}$$

while u still consumes capacity, adds an admissible route, or contributes to inconsistent updates. Such a vertex is not merely inactive. It is structurally dangerous when it expands the space of possible flows without increasing task information.

A useful pruning score has the schematic form

$$S(u) = \Delta I(C; Y \mid u) - \lambda\, \mathrm{CapCost}(u) - \mu\, \mathrm{InconsistencyRisk}(u). \tag{58}$$

Vertices with small or negative score are candidates for removal.

5.3 Pruning as a useful entropy constraint

Pruning is commonly interpreted as compression or regularization. In the present framework, its stronger role is to create a useful entropy constraint. Removing unnecessary vertices reduces the number of admissible routes through which inconsistent information can enter. It also concentrates the available conditional entropy capacity around validated task-relevant flows.

Let $\mathcal{G}$ be the original graph and $\mathcal{G}'$ the pruned graph. The desired pruning operation satisfies

$$I(C; Y_{\mathcal{G}'}) \approx I(C; Y_{\mathcal{G}}) \tag{59}$$

while reducing irrelevant capacity:

$$C_{\mathrm{irr}}(\mathcal{G}') < C_{\mathrm{irr}}(\mathcal{G}). \tag{60}$$

Then the ratio of task-relevant flow to irrelevant admissible capacity increases:

$$\frac{\Phi^{\mathrm{rel}}_{\mathcal{G}'}}{C^{\mathrm{irr}}_{\mathcal{G}'}} > \frac{\Phi^{\mathrm{rel}}_{\mathcal{G}}}{C^{\mathrm{irr}}_{\mathcal{G}}} \tag{61}$$

whenever task-relevant flow is preserved and irrelevant capacity is sufficiently reduced.

This explains why reducing or pruning unnecessary vertices can improve robustness and prevent inconsistency. It does not make the processor more powerful in a purely expressive sense. It makes the finite-capacity competition sharper and more meaningful.

5.4 Small parameter space

For a small θ-space, unnecessary vertices are particularly costly. The machine has limited degrees of freedom, so redundant or irrelevant routes compete directly with necessary routes. In this regime, pruning is not an optional efficiency trick. It is part of maintaining a consistent information geometry: the graph should contain enough structure to solve the task, but not so much unused structure that false constraints can be absorbed without conflict.

6 Reasoning circulation and logical-gap repair as requirements of robustness

6.1 Sequential reasoning as supervised sequence learning

A reasoning trace can be represented as a sequence

$$R_0 \to R_1 \to \cdots \to R_m, \tag{62}$$

where each R_k is a statement, representation, or intermediate conclusion. Supervised learning of reasoning is then supervised learning of sequential outputs: the model is trained not only to produce the final answer but also to produce intermediate transitions.

However, merely copying a given chain does not guarantee understanding of the logical transitions. A gap exists when the transition $R_k \to R_{k+1}$ has insufficient conditional support.

6.2 Logical-gap uncertainty

Define the gap uncertainty

$$G_k := H(R_{k+1} \mid R_k, \mathcal{B}_k), \tag{63}$$

where $\mathcal{B}_k$ denotes the currently available background information. A strong transition has small G_k. A weak transition has large G_k.

A bridge variable B_k repairs the gap if

$$I(B_k; R_{k+1} \mid R_k, \mathcal{B}_k) > 0 \tag{64}$$

and

$$H(R_{k+1} \mid R_k, \mathcal{B}_k, B_k) < H(R_{k+1} \mid R_k, \mathcal{B}_k). \tag{65}$$

Thus strengthening a logical gap means strengthening the bridge across the gap, not increasing the unresolved uncertainty itself. The unresolved gap produces a demand for information; the validated bridge supplies task-relevant information.

6.3 Endogenous questioning

The machine can be said to "feel" a gap in the operational sense that a transition produces high conditional uncertainty, high conflict among candidate continuations, or low validation score. It can then generate a question internally:

$$Q_k = q(R_k, R_{k+1}, \mathcal{B}_k), \tag{66}$$

obtain or generate a candidate bridge B_k, validate it, and insert it into the reasoning route. This is a local version of the same circulation:

$$\text{gap detection} \to \text{bridge proposal} \to \text{validation} \to \text{reasoning update}. \tag{67}$$

The information flow in the reasoning circulation increases when the bridge is validated:

$$\mathcal{T}_{B_k \to R_{k+1} \mid R_k, \mathcal{B}_k} > 0. \tag{68}$$

Therefore, repairing a logical gap increases task-relevant information flow in the circulation. This makes reasoning improvement a special case of self-learning under validation.

7 Multitask learning and context-dependent effective subgraphs

7.1 Context-conditioned effective graph

Let M denote a task context or problem type. A fixed parameter graph $\mathcal{G}_\theta$ may produce different effective graphs depending on M:

$$\mathcal{G}_{\text{eff}}(M) = (\mathcal{V}_{\text{eff}}(M), \mathcal{E}_{\text{eff}}(M)). \tag{69}$$

A vertex or edge is effective for context M if it carries non-negligible task-relevant operational flow:

$$v \in \mathcal{V}_{\text{eff}}(M) \quad \Longleftrightarrow \quad \Phi_v^{\text{rel}}(M) > \epsilon. \tag{70}$$

Different tasks may therefore use the same underlying parameters but activate different subgraphs.

7.2 Shared and exclusive components

For two task contexts M_1 and M_2, define

$$\mathcal{G}_{\text{shared}} = \mathcal{G}_{\text{eff}}(M_1) \cap \mathcal{G}_{\text{eff}}(M_2), \tag{71}$$

$$\mathcal{G}_{1\setminus 2} = \mathcal{G}_{\text{eff}}(M_1) \setminus \mathcal{G}_{\text{eff}}(M_2), \qquad \mathcal{G}_{2\setminus 1} = \mathcal{G}_{\text{eff}}(M_2) \setminus \mathcal{G}_{\text{eff}}(M_1). \tag{72}$$

Shared subgraphs encode reusable features or operations. Context-specific subgraphs encode task-dependent transformations, answer formats, or constraints. These subgraphs need not be exclusive. In a sufficiently large or high-capacity graph, several task-dependent subgraphs may remain active at the same time and overlap broadly. Partial exclusivity emerges only when simultaneous activation causes capacity competition or representational incompatibility.

Partially exclusive subgraphs emerge naturally when the following conditions hold:

1. task contexts require different conditional distributions $P(C \mid X, M)$;
2. some internal routes useful for one task interfere with another;
3. finite capacity prevents all routes from being simultaneously active without loss;
4. a context variable or gating mechanism can modulate the effective graph.

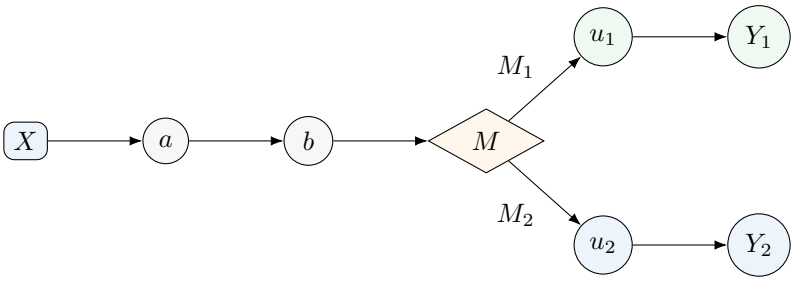
Under these conditions, forcing a single fully shared route creates interference. Conditional subgraph activation reduces interference by separating incompatible flows while preserving shared routes where they are genuinely reusable. Without these conditions, multitask learning can be realized by overlapping effective subgraphs rather than exclusive modules.

7.3 Deep layers and task classification

A machine that faces many types of problems with the same parameters benefits from deep organization because early layers can extract broadly reusable information while later layers can classify the problem type and route information into task-specific subgraphs. This is not merely an empirical convenience. In the information-dynamical setting, the deeper structure supports a separation between

$$\text{shared task-relevant flow} \qquad \text{and} \qquad \text{context-specific task-relevant flow}. \tag{73}$$

The task classifier itself becomes a routing vertex. Its output controls which downstream subgraph receives capacity.



8 Compatibility between robustness and multitasking

The robustness argument and the multitask argument point in different directions, but they are not fundamentally contradictory. Robustness favors constrained admissible flow: unnecessary routes

are removed so that inconsistent information must compete with the validated internal circulation. Multitasking favors context-dependent admissible flow: the same finite processor must preserve enough routes to solve several classes of problems. The apparent conflict appears only if robustness is interpreted as globally minimal pruning.

For a single task, a compact task-sufficient graph may be written schematically as $\mathcal{G}_{\text{task}}$. For several task contexts $M \in \mathcal{M}$, the retained graph should instead preserve the union of the shared and context-specific effective graphs:

$$\mathcal{G}_{\text{retain}} = \mathcal{G}_{\text{shared}} \cup \bigcup_{M \in \mathcal{M}} \mathcal{G}_{\text{specific}}(M). \tag{74}$$

Thus multitask learning changes the meaning of "unnecessary". A vertex is unnecessary only when its incremental task-relevant contribution is negligible across the relevant contexts and when it is not required for a shared abstraction, a context-specific route, a reasoning bridge, or a robustness-critical backup path:

$$\Delta_v^{\text{rel}}(M) \simeq 0 \quad \text{for all } M \in \mathcal{M}. \tag{75}$$

Pruning outside $\mathcal{G}_{\text{retain}}$ can strengthen robustness, whereas pruning inside this union can damage multitask competence.

The capacity condition makes the compatibility precise. If two task flows A_{M_1} and A_{M_2} can be jointly represented at a vertex v without exceeding its entropy capacity,

$$I(A_{M_1}, A_{M_2}; Z'_v \mid Z_v) \le C_v, \tag{76}$$

then the corresponding effective subgraphs may coexist and overlap. If instead

$$I(A_{M_1}, A_{M_2}; Z'_v \mid Z_v) > C_v, \tag{77}$$

then simultaneous activation requires displacement, interference, or reorganization. In that case the robust multitask architecture must introduce routing, gating, or partially exclusive subgraphs. Therefore, robustness and multitasking are compatible when the entropy constraint is context-aware rather than globally minimal.

The resulting design principle is:

$$\begin{aligned} \text{robust multitask learning} = {} & \text{validated circulation} + \text{context-aware pruning} \\ & + \text{conditional routing under finite capacity}. \end{aligned} \tag{78}$$

9 Assumption map

The note uses several assumptions. They should be kept explicit.

Assumption	Role	What changes if relaxed
Finite capacity	Produces competition among incoming flows	Without capacity limits, inconsistent flows may be stored in unused degrees of freedom rather than forced to compete
Graph-structured computation	Allows operational TE to be localized and recursively propagated	In a fully black-box representation, the recursion and pruning criteria lose structural meaning

Two timescales	Separates operational propagation from learning updates	If weights change during operation, operational and learning TE must be analyzed jointly
Feedforward operational subgraph	Enables topological recursion	Recurrent graphs require cycle-level or stationary-flow analysis
Validation of generated pairs	Prevents self-learning from amplifying false constraints	Without validation, internal generation can destabilize the model
Higher-order monitoring at mergers	Detects redundancy and synergy	Pairwise TE alone can misrepresent joint information flow
Task-relevance criterion	Distinguishes useful flow from raw activity	Without it, pruning may remove useful hidden structure or preserve irrelevant activity

10 Limits and non-claims

This note is a structural theory, not yet a complete algorithm. It identifies what kind of architecture is forced by the problem setting, but it does not specify a unique optimizer, estimator, or neural implementation.

The following claims are not made.

1. The note does not claim that every learning system requires circulation. The claim is restricted to sustained label-efficient self-learning with validation.
2. The note does not claim that all external information competing with circulation is false. Compatible information can enter as redundant or confirmatory flow.
3. The note does not claim that pruning always improves performance. Pruning is useful when it removes irrelevant capacity while preserving task-relevant circulation.
4. The note does not claim that pairwise TE is sufficient. At merging vertices, higher-order information can be essential.
5. The note does not claim that multitask subgraphs must be completely disjoint. The expected structure is partially shared and partially exclusive.

11 Open theoretical directions

Several directions remain open.

1. Develop estimators for operational and learning TE that remain stable in high-dimensional parameter graphs.
2. Derive pruning algorithms directly from changes in task-relevant operational influx Φ_v^{rel}.
3. Extend the feedforward recursion to recurrent graphs by decomposing cycle circulations and transient operational flows.
4. Formalize validation as a capacity-constrained hypothesis test between internal circulation and external input.
5. Relate task-conditioned effective subgraphs to a measurable information-flow hypergraph.
6. Study how bridge generation in reasoning can be optimized by maximizing the reduction of logical-gap entropy.

12 Conclusion

Starting from a minimal supervised-learning setting, the note derives a coherent design logic for an information processor that is efficient, robust, self-consistent, and capable of multitask operation. The key transition is from a one-way supervised feed to a validated circulation. Once the machine is required to continue learning with limited external labels, internally generated constraints must be proposed, validated, and returned to the parameter dynamics. Once finite entropy capacity is imposed, validated circulation competes with incompatible new flows. Once the machine is graph-structured, feedforward operational flow can be recursively calculated, and higher-order terms identify redundancy and synergy at merging vertices. Once unnecessary vertices are removed, the remaining graph becomes a sharper entropy-constrained processor. Once multiple tasks are present, the same graph naturally separates into shared and context-dependent effective subgraphs.

The resulting picture is not that intelligence is merely a larger approximator. It is that a robust learning machine must organize information flow, validation, capacity, pruning, reasoning repair, and multitask routing into one self-consistent information-dynamical system.

A Exact identities used in the note

A.1 Transfer-entropy balance identity

For arbitrary random variables A, Z, Z', the chain rule gives

$$I(A;Z') - I(A;Z) = \big[H(A) - H(A \mid Z')\big] - \big[H(A) - H(A \mid Z)\big] \tag{79}$$
$$= H(A \mid Z) - H(A \mid Z'). \tag{80}$$

Add and subtract $H(A \mid Z, Z')$:

$$I(A;Z') - I(A;Z) = \big[H(A \mid Z) - H(A \mid Z, Z')\big] - \big[H(A \mid Z') - H(A \mid Z, Z')\big] \tag{81}$$
$$= I(A;Z' \mid Z) - I(A;Z \mid Z'). \tag{82}$$

Thus

$$\Delta I(A;Z) = \mathcal{T}_{A\to Z} - \mathcal{T}^{\mathrm{rev}}_{A\to Z}. \tag{83}$$

A.2 Two-source interaction decomposition

For two sources A, B and target update Z', conditioned on Z,

$$I(A,B;Z' \mid Z) = I(A;Z' \mid Z) + I(B;Z' \mid Z, A), \tag{84}$$
$$I(A;B;Z' \mid Z) = I(A;Z' \mid Z) - I(A;Z' \mid Z, B). \tag{85}$$

Eliminating the conditional term gives

$$I(A,B;Z' \mid Z) = I(A;Z' \mid Z) + I(B;Z' \mid Z) - I(A;B;Z' \mid Z). \tag{86}$$

The sign of $I(A;B;Z' \mid Z)$ distinguishes redundancy and synergy under the interaction-information convention.

A.3 Residual-capacity bound

From

$$I(A,B;Z'_v \mid Z_v) \le C_v \tag{87}$$

and

$$I(A, B; Z'_v \mid Z_v) = I(A; Z'_v \mid Z_v) + I(B; Z'_v \mid Z_v, A), \tag{88}$$

one obtains

$$I(B; Z'_v \mid Z_v, A) \leq C_v - I(A; Z'_v \mid Z_v). \tag{89}$$

If $I(A; Z'_v \mid Z_v) \geq C_v - \varepsilon$, then

$$I(B; Z'_v \mid Z_v, A) \leq \varepsilon. \tag{90}$$

B Recursive feedforward template

Given a feedforward graph:

1. Choose receiving vertices R and output vertices O.
2. Compute boundary flows $\Phi_r = \mathcal{T}_{X \to r}$ for $r \in R$.
3. Traverse the graph in topological order.
4. At each internal vertex v, compute

$$\Phi_v = \sum_{p \in \mathrm{Pa}(v)} \Phi_p - E_{\mathrm{Pa}(v) \to v}. \tag{91}$$

5. Propagate Φ_v to each child at first activation.
6. At each output vertex o, read Φ_o as the graph-to-output operational influx.

This template makes clear where redundancy, synergy, pruning, and task routing enter: they all modify either the parent set $\mathrm{Pa}(v)$, the correction $E_{\mathrm{Pa}(v) \to v}$, or the boundary/task condition.

C Toy examples

C.1 Single chain

For

$$X \to v_1 \to v_2 \to \cdots \to o, \tag{92}$$

each internal vertex has one parent. Therefore all higher-order corrections vanish and

$$\Phi_o = \mathcal{T}_{X \to v_1}. \tag{93}$$

The chain transmits operational TE with delay but without merger correction.

C.2 Redundant parents

If p_1 and p_2 are correlated and both inform v, then

$$\Phi_v = \mathcal{T}_{p_1 \to v} + \mathcal{T}_{p_2 \to v} - \mathcal{T}^2_{\{p_1, p_2\} \to v}. \tag{94}$$

For positive $\mathcal{T}^2$, the pairwise sum overcounts the joint contribution. Pruning one parent can preserve output task information if the removed parent contributes only redundant flow.

C.3 Synergistic parents

For an XOR-like merge, each separate parent may have small ordinary TE while the joint source is informative. Then the hyperarc $\{p_1, p_2\} \to v$ must be preserved. Pairwise pruning would be harmful because it would destroy the group interaction.

C.4 Near-capacity merge

If an established internal circulation A satisfies

$$I(A; Z'_v \mid Z_v) = C_v - \varepsilon, \tag{95}$$

then any competing source B satisfies

$$I(B; Z'_v \mid Z_v, A) \leq \varepsilon. \tag{96}$$

This gives the cleanest mathematical expression of robustness by capacity saturation.

C.5 Two-task gate

Suppose a context variable M routes a shared representation into either u_1 or u_2. Then

$$\mathcal{G}_{\text{eff}}(M_1) \neq \mathcal{G}_{\text{eff}}(M_2), \tag{97}$$

while both may share the upstream representation. This is the simplest model of partially exclusive subgraphs.

D Glossary

Term	Meaning
Task information	Mutual information between the correct answer C and output Y, usually $I(C;Y)$
Operational TE	Transfer entropy through the fixed computational graph during fast operation
Learning TE	Transfer entropy that modifies parameters across slow learning steps
Multiple TE	Interaction-information contribution of a source set to a target update
Information circulation	Closed validated route in which learned structure generates, tests, and reinserts constraints
Residual capacity	Remaining conditional entropy capacity after an existing flow is accounted for
Logical gap	A reasoning transition with high conditional uncertainty or low validation support
Bridge information	Additional variable or step that reduces logical-gap uncertainty
Pruning	Removal of unnecessary vertices or edges while preserving task-relevant flow
Effective subgraph	Context-dependent active part of a fixed parameter graph